

ALGORITHM XXXX: PSM4ODES: AUTOMATIC SERIES RECURRENCE RELATIONS FOR ORDINARY DIFFERENTIAL EQUATIONS

RICHARD D. NEIDINGER
DAVIDSON COLLEGE, NC, USA
RINEIDINGER@DAVIDSON.EDU

Abstract. An automatically generated recurrence algorithm can compute the coefficients of the power series solution of an ordinary differential equation(s) (ODEs) as given in MATLAB for numerical solution. The algorithm has a variable initial point to any variable order, allowing large step sizes in a numerical high-order Taylor series method. This algorithmic approach has been known for decades but is introduced and implemented here for a broad audience. A program writes the MATLAB code for the recurrence algorithm, clearly readable evidence that power series solutions are possible for almost any (including nonlinear) ODEs. This gives a high-order Taylor polynomial solution that is good within much of the radius of convergence. Tests show that stepping produces piecewise polynomial solutions that can achieve high accuracy with far fewer steps than usual methods, `ode45` in particular. The program that writes the recurrence uses operator overloading in object-oriented programming in MATLAB.

1. Introduction. This article presents an introduction to a modern algorithmic approach to power series solutions for differential equations and includes implementation details for the numerical method. We focus on forming, viewing, and then using the recurrence algorithm for series coefficients in both pseudocode and in automatically generated MATLAB code. The generating program writes a series recurrence relation, as an algorithm, for an ordinary differential equation (ODE) that can be written as a system of first order ODEs (as is typically done for numerical solutions), about an arbitrary initial condition. We seek to broaden awareness and understanding, accessible to undergraduates and researchers in related fields, of this algorithmic Taylor method that has been around for decades in different versions and that researchers have found to be useful (e.g. [2] [3]). The method contrasts with many traditional treatments of power series solutions in differential equations (e.g. [5]) and of higher-order Taylor methods in numerical analysis (e.g. [6]) that still give the impression that symbolic manipulations are required, either by hand or using symbolic computation. We use the terms power series and Taylor series interchangeably with the understanding that expansion about an arbitrary point is necessary for steps in an ODE solver.

The algorithmic Taylor method dates back to at least 1960 [10] and is well-known in the automatic differentiation research community [11] [16] [26]. Specialized software tools, for solving ODEs using this method, date from the last century [7] [8] to more recent times [1] [3] [13] [17]. Implementations can be complex, relying on either source transformation (reading and writing text files for an ODE and series respectively) or object-oriented programming (OOP) that overloads the operations in the ODE. Another research community [4] [23] [29] [32] has taken a related approach to bridging the gap between an ODE and the recurrence relation algorithm for the power series solution. They focus on rewriting any ODE system as a system of only polynomial (or just quadratic) expressions for each derivative, as we mention in Section 2.

The accompanying software package PSM4ODES (Power Series Method For ODEs) in MATLAB uses OOP to accomplish source code transformation from the same file that would be used for built-in solvers such as `ode45`, into a file that generates series coefficients for the solution about any point. MATLAB is already used in many

numerical analysis classes and other applied settings, and has been used for efficient implementations of automatic differentiation to compute derivatives of functions defined by computer code, both using OOP directly [9] and using OOP to perform source code transformation [24]. Future versions of the PSM4ODES software will be made available via [21].

There are a few restrictions on a system of ODEs that will work in PSM4ODES. First, we assume that the system has a unique smooth solution in a neighborhood of the intended initial point(s). Second, the system should be written in form $y' = f(t, y)$ for vector y of functions, where components of f are composed of operations and standard functions as found on scientific calculators. Specifically, our method of generating recurrence relations requires a simple derivative chain rule as described in the next paragraph, ruling out a Gamma function for example. Third, avoid branches in the coding of f since the recurrence code is generated by one execution of f , and this code is only valid if input values go through the same sequence of operations and functions. Fourth, f may use only scalar functions, for the purpose of simplicity in this package. In MATLAB code for f , vector (or matrix) functions such as `.*` and `.^` must be rewritten as loops, so that each scalar operation executed will generate lines in the series recurrence code, as discussed in the Pleiades example in Section 7. The generated recurrence code, and programming in PSM4ODES, do use efficient vector and matrix operations, based off of scalar operations in f . Finally, the code for f may not preallocate the output vector (for y') with constants, although preallocating output vector equal to input vector is fine. This allows for overloading input y and getting the same class of output.

Since products and quotients of power series have recurrence principles, such principles extend to standard functions that satisfy equations of the form $w' = vu'$ or $w' = u'/v$, as explained in Sections 2 and 3. For the simplest example, $w = \exp(u)$ satisfies $w' = wu'$. Other standard function recurrences come from one of, or a pair of equations, of the form $w' = vu'$ or $w' = u'/v$. Conceptually, we convert the system $y' = f(t, y)$ into a system of particularly simple Differential-Algebraic Equations (DAEs), that we call *arithmetic DAEs* (defined in Section 3), although these are only used as an organized way to code series recurrence relations for each intermediate step in the computation of f . A more general approach by Nedialkov and Pryce in [17] (and their papers referenced therein) succeeds in computing Taylor series to solve systems of DAEs. Based on Pryce's structural analysis [25], they prove sufficient conditions on the DAEs so that method will succeed. In a contemporary paper [18], they introduce a more general idea of a sub-ODE that plays much the same role as our equations of form $w' = vu'$ or $w' = u'/v$. Their software executes steps based upon a coded list something like our conceptual list of arithmetic DAEs, whereas we generate and execute code that directly computes Taylor series for the ODE about a point, with numeric input and output as in $f(t, y)$. They theoretically prove that the recurrence relations work, so that no Taylor coefficient is required before it has been computed.

The sections in this paper can be used to appreciate the algorithmic Taylor method on several different levels. Section 2 is an elementary introduction, that could be used as an alternative way to present power series solutions in differential equations. Section 3 provides a summary of the foundational recurrences behind the method [11] [16] [20] [31]. This section can be skipped or skimmed and returned to as a reference as needed. Section 4 focuses on the output MATLAB code, that is, the series recurrence algorithm for a given ODE at variable initial condition to vari-

able order. This tangible result could help develop appreciation for a computational approach to power series solutions or the high-order Taylor method. Practically, executing this code is more efficient (and would be even more so in a compiled language) than an alternative that directly overloads the operators to perform (instead of write) the recurrence step(s). Seeing the recurrence code can help bridge the gap in many black-box solvers from specifying an ODE to the computed solution. Section 5 shows how object-oriented programming with operator overloading is used to turn operations done in the ODE into character strings for each recurrence that are assembled into the output series recurrence file. This section can be skipped for readers interested in using, not implementing, the method. However, such details may be a highlight for those interested in computing details that are often omitted with other packages. The ideas could be used in other languages, perhaps Python for an alternative exposition or C++ for computational efficiency. In fact, by changing text output, our MATLAB code could write series code in other languages. Section 6 presents ODE solvers, fixed and adaptive step, that use the generated series recurrence code. Illustrative examples are provided with accuracy and efficiency comparison of solvers, including `ode45`. While generating and using series code is our emphasis, Section 7 focuses on the performance of our adaptive step solver `odepsmJZ` relative to several MATLAB solvers, including `ode89`. We report using work-precision diagrams as popularized by the *Test Set for IVP Solvers* [14] [15], the source of a couple of the example problems. Using high-order series, our solver does achieve good high-precision results using fewer steps than the MATLAB solvers, but it is usually slower.

2. First examples and principles. All functions will be assumed to be real analytic in an open interval about an initial condition. So every function $y(t)$ and initial t_0 corresponds to a theoretically infinite array Y of Taylor series coefficients:

$$y(t) = Y[0] + Y[1](t - t_0) + Y[2](t - t_0)^2 + \cdots = \sum_{k=0}^{\infty} Y[k](t - t_0)^k.$$

For clarity of exposition and pseudo-code, we use index origin zero for the array Y and index into it with square brackets to emphasize the transformation from a function evaluation $y(t)$ to an array entry $Y[k]$. Any function about t_0 will correspond to an array, so a function u corresponds to array U , and likewise with any letter denoting a function of t .

For a preliminary example, consider Airy's equation with initial values

$$y'' - ty = 0; \quad y(t_0) = a; \quad y'(t_0) = b.$$

As is done for numerical solution, use $y_1 = y$ and $y_2 = y'$ and rewrite the system $y'_1 = y_2$ and $y'_2 = ty_1$. In a programming environment, the user then defines the function $f(t, (y_1, y_2)) = (y_2, ty_1)$. In an algorithm to write a recurrence relation, each operation in f will correspond to an auxiliary variable (function) assignment, here $u_1 = ty_1$, together with the system $y'_1 = y_2$ and $y'_2 = u_1$ that assigns one of the variables to each derivative.

The variables and functions t, y_1, y_2, u_1 have corresponding arrays of series coefficients T, Y_1, Y_2, U_1 . Initial values specify $Y_1[0] = a$ and $Y_2[0] = b$. Since $t = t_0 + (t - t_0)$, the array T is completely known, $T[0] = t_0$, $T[1] = 1$, and $T[k] = 0$ for $k \geq 2$. The U and Y arrays can be updated by comparing series terms in the

assignments. Substituting series into the assignment $u_1 = t y_1$, yields

$$\begin{aligned} \sum_{k=0}^{\infty} U_1[k](t - t_0)^k &= (T[0] + T[1](t - t_0)) \left(\sum_{k=0}^{\infty} Y_1[k](t - t_0)^k \right) \\ &= \sum_{k=0}^{\infty} T[0]Y_1[k](t - t_0)^k + \sum_{j=1}^{\infty} T[1]Y_1[j-1](t - t_0)^j, \end{aligned}$$

resulting in our first recurrence step

$$U_1[k] = T[0]Y_1[k] + T[1]Y_1[k-1] \text{ for } k \geq 1.$$

To update Y using the system $y'_1 = y_2$ and $y'_2 = u_1$, we consider any equation of the form $y'_i = v$, which we call a **DE assignment**, where v is the name of one function of t . Then series

$$\begin{aligned} y'_i &= \sum_{k=0}^{\infty} V_i[k](t - t_0)^k \text{ and} \\ y'_i &= \sum_{k=1}^{\infty} kY_i[k](t - t_0)^{k-1} = \sum_{k=0}^{\infty} (k+1)Y_i[k+1](t - t_0)^k. \end{aligned}$$

Equating coefficients in each series gives $(k+1)Y_i[k+1] = V_i[k]$ which is the principle to update Y .

PRINCIPLE 2.1 (DE assignment and Y update). *For $k \geq 0$,*

$$\text{if } y'_i = v, \text{ then } Y_i[k+1] = V_i[k]/(k+1).$$

Starting from initial values, series coefficients for the solution of Airy's equation can be generated from the following summary table.

operation auxiliary	auxiliary initialization	recurrence steps
$u_1 = t y_1$	$U_1[0] = T[0]Y_1[0]$	$U_1[k] = T[0]Y_1[k] + T[1]Y_1[k-1]$
DE assignments	update Y	
$y'_1 = y_2$	$Y_1[1] = Y_2[0]$	$Y_1[k+1] = Y_2[k]/(k+1)$
$y'_2 = u_1$	$Y_2[1] = U_1[0]$	$Y_2[k+1] = U_1[k]/(k+1)$

This shows the structure of our algorithmic Taylor method, executing first down the second column, then repeating down the third column for $k \geq 1$.

To compare to classic series recurrence for $y = y_1$ in a DE course, use middle recurrence step above to form $Y_1[k+2] = Y_2[k+1]/(k+2)$, then substitute for this $Y_2[k+1]$ using the bottom step, to get $Y_1[k+2] = U_1[k]/((k+1)(k+2))$, and finally, by top step, $Y_1[k+2] = (t_0 Y_1[k] + Y_1[k-1])/((k+1)(k+2))$. This agrees with the simpler cases found in [5], Section 5.2, where graphs and coefficient patterns are shown. For such simple DEs and assumptions, we could streamline derivation of simple recurrences, but our goal is to generalize and automate! Keeping arbitrary t_0 allows for the same recurrence program to work for each step of a piecewise series solution, using steps as in usual numerical solutions but giving polynomial approximations of the series solutions between relatively large steps.

A nonlinear DE will motivate the two key principles behind recurrence relations for all other nonlinear operations. This example

$$y' = \sin(y^2) \text{ with } y(t_0) = a \quad (\text{Example 1})$$

will guide exposition in subsequent sections. For the squaring operation, we consider the more general case of multiplication of two functions and corresponding multiplication of series. A product series converges as long as one of the original series is absolutely convergent (within the radius of convergence) and the other is convergent. To compute the k th coefficient of the product requires previously computed coefficients 0 to k for the multiplied functions, making a computer algorithm more practical than simple hand recurrence relations in most DE books. Specifically, suppose $w(t) = u(t)v(t)$, then

$$\sum_{k=0}^{\infty} W[k](t-t_0)^k = \left(\sum_{i=0}^{\infty} U[i](t-t_0)^i \right) \left(\sum_{j=0}^{\infty} V[j](t-t_0)^j \right).$$

For the multiplication to result in a specific power $(t-t_0)^k$, we need all i and j with $i+j=k$. This results in the following well-known principle called the Cauchy product.

PRINCIPLE 2.2 (Cauchy product). *For $k \geq 0$,*

$$\text{if } w(t) = u(t)v(t), \text{ then } W[k] = \sum_{i=0}^k U[i]V[k-i].$$

For the $\sin(u)$ operation, we define two variables or auxiliary functions $s = \sin(u)$ and $c = \cos(u)$. Then $s' = c u'$ and $c' = -s u'$. With series adjusted for the derivative, this pair of multiplications can be computed by a Cauchy product. In general, suppose $w'(t) = \alpha u'(t)v(t)$ where α is a constant, so

$$\sum_{k=1}^{\infty} kW[k](t-t_0)^{k-1} = \alpha \left(\sum_{i=1}^{\infty} iU[i](t-t_0)^{i-1} \right) \left(\sum_{j=0}^{\infty} V[j](t-t_0)^j \right).$$

Comparing coefficients of $(t-t_0)^{k-1}$, we still need terms with $i+j=k$. The resulting principle uses previously computed coefficients 1 to k for u but only 0 to $k-1$ for v , so function v may be w itself or even a later function in the list of operation auxiliaries as in (2.1) below.

PRINCIPLE 2.3 (derivative Cauchy product). *For $k \geq 1$,*

$$\text{if } w'(t) = \alpha v(t)u'(t), \text{ then } W[k] = \frac{\alpha}{k} \sum_{i=1}^k iU[i]V[k-i].$$

For the nonlinear ODE $y' = \sin(y^2)$, let $y_1 = y$ and initialize $Y_1[0] = y(t_0) = a$. Conceptually, evaluation of $\sin(y^2)$ generates a list of auxiliary variables, one for the operation y^2 and two for $\sin$, followed by the DE assignment, as shown in the following

table. The second column replaces the transcendental functions with an equation of the form of the hypothesis of Principle 2.3.

operation auxiliaries	arithmetic DAEs
$u_1 = y_1 y_1$	$u_1 = y_1 y_1$
$u_2 = \sin(u_1)$	$u'_2 = u_3 u'_1$
$u_3 = \cos(u_1)$	$u'_3 = -u_2 u'_1$
DE assignment	
$y'_1 = u_2$	$y'_1 = u_2$

(2.1)

The set of arithmetic DAEs is an alternative way to describe the original ODE, using only operations that match one of our Principles (more about this in the next section). This conceptual table corresponds directly to the initialization and recurrence steps given by these Principles.

auxiliary initializations	recurrence steps
$U_1[0] = Y_1[0]Y_1[0]$	$U_1[k] = \sum_{i=0}^k Y_1[i] Y_1[k-i]$
$U_2[0] = \sin(U_1[0])$	$U_2[k] = \frac{1}{k} \sum_{i=1}^k i U_1[i] U_3[k-i]$
$U_3[0] = \cos(U_1[0])$	$U_3[k] = \frac{-1}{k} \sum_{i=1}^k i U_1[i] U_2[k-i]$
update Y	
$Y_1[1] = U_2[0]$	$Y_1[k+1] = U_2[k]/(k+1)$

(2.2)

This process of creating recurrence steps generalizes to almost any ODE. Most standard functions (like $\sin$) have derivatives that can be written as just one or two equations matching Principles above, or similar quotient versions, as shown in Section 3. A program can automate this process to take the DE function and directly write and/or implement code for the recurrence steps. Creating, by hand, a conceptual table of the arithmetic DAEs and/or subsequent recurrences is a good exercise in learning this method and can be useful if you are primarily interested in one differential equation and want to hand-code a specific solution. In any case, computer execution of code is preferred to generate the arbitrarily high series coefficients for the solution of the ODE about any t_0 , given the initial value.

An alternative approach uses polynomial systems, as in [29]. For the above example, the derivative products could include $u'_1 = 2yy' = 2y_1 u_2$ and this could be substituted into the equations u'_2 and u'_3 , yielding a polynomial system. Recurrence steps can be formed using only the Cauchy product rule instead of specialized rules for each function, which may be pedagogically simpler. Theoretical error bounds have been derived using this method [32] [30] which can be generalized to our algorithms. The polynomial system approach is augmented by recurrences for scientific functions in [12].

To show an application and the efficacy of the recurrence steps in (2.2) above, they produce the Taylor polynomial solutions shown in Figure 2.1 using $y(0) = 0.1$. One solution curve was generated by the usual MATLAB solver `ode45`. The recurrences generated coefficients for one Maclaurin polynomial of degree 25, also shown for degrees 5 and 10. The last curve of points uses the same recurrence steps to degree 12 but restarting with each marked point as chosen by an adaptive step size algorithm discussed in Section 6; accomplishing error within 10^{-9} using only 33 steps. In comparison, `ode45` with requested tolerance of 10^{-9} used 119 steps.

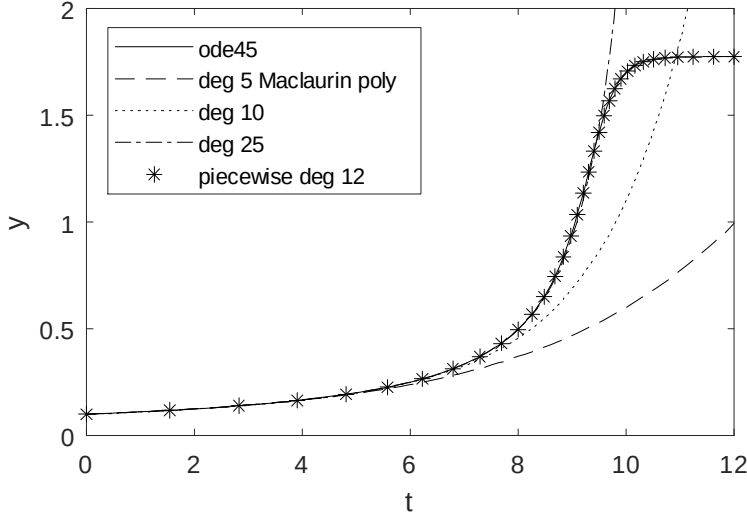


FIG. 2.1. Plots using recurrence algorithm for solution of $y' = \sin(y^2)$

3. Recurrences for other operations and standard functions. Operations for any linear combinations can be directly reflected as a simple recurrence step.

If $w(t) = u(t) \pm v(t)$, then $W[k] = U[k] \pm V[k]$.

If $w(t) = \alpha u(t)$, then $W[k] = \alpha U[k]$.

If $w(t) = u(t) + \alpha$, then $W[0] = U[0] + \alpha$ and $W[k] = U[k]$ for $k \geq 1$.

Division is just a Cauchy product solved for the quotient coefficient. If $w(t) = u(t)/v(t)$, then $u(t) = w(t)v(t)$, so by Cauchy product $U[k] = \sum_{i=0}^k W[i] V[k-i] = W[k]V[0] + \sum_{i=0}^{k-1} W[i] V[k-i]$. Solving for $W[k]$ yields the following that uses W only up to $k-1$.

PRINCIPLE 3.1 (Cauchy quotient). For $k \geq 1$,

$$\text{If } w(t) = u(t)/v(t), \text{ then } W[k] = \left(U[k] - \sum_{i=0}^{k-1} W[i] V[k-i] \right) / V[0].$$

Similarly, if $w'(t) = \alpha u'(t)/v(t)$, then $u'(t) = (1/\alpha)w'(t)v(t)$, so by Principle 2.3, $U[k] = \frac{1/\alpha}{k} \sum_{i=1}^k iW[i] V[k-i]$. Then

$$\alpha U[k] = W[k]V[0] + \frac{1}{k} \sum_{i=1}^{k-1} iW[i] V[k-i].$$

Solving for $W[k]$ gives a principle that uses W and V only up to $k-1$. The resulting principle uses previously computed coefficient $U[k]$ but only indices 0 to $k-1$ for v or w , so function v may be w itself or even a later function in the list of operation auxiliaries.

PRINCIPLE 3.2 (derivative Cauchy quotient). For $k \geq 1$,

$$\text{if } w'(t) = \alpha u'(t)/v(t), \text{ then } W[k] = \left(\alpha U[k] - \frac{1}{k} \sum_{i=1}^{k-1} iW[i] V[k-i] \right) / V[0]$$

Now our set of principles is sufficient to handle almost any ODE system typically encountered. Define an **arithmetic DAE** to be an equation is of the form $w = \alpha u$, $w = u + v$, $w = u - v$, $w = uv$, $w = u/v$, $w' = \alpha v u'$, $w' = \alpha u'/v$, or $y' = u$ (as in Principle 2.1). (Form $w = \alpha u$ includes negation and division by scalar.) Conceptually, each operation, including standard functions (as on a scientific calculator), in the evaluation of the ODE system becomes one to three arithmetic DAEs and each of these has a corresponding recurrence. Linear operations, products, and quotients, each become just the corresponding algebraic equation in the arithmetic DAE system, and the corresponding recurrence is either the trivial linear one or given in Principle 2.2 or 3.1. We claim that the derivative of most other standard functions satisfy either one or two arithmetic DAEs of the form in Principle 2.3 or 3.2, with corresponding recurrences given therein. Nine of the most common examples are given in Table 3.1, with others discussed in the last two paragraphs of this Section.

These recurrences work (are consistent and not circular) if the right side of the arithmetic DAE uses only functions where the k th coefficient is already known (earlier on the DAE list or y), with the important exception that v (not the derivative) in Principles 2.3 and 3.2 can be any auxiliary function in the DAE. In Table 3.1, the latter operations all use a pair of arithmetic DAEs, where the first uses the second auxiliary, and the second (without derivative) may use the first as either operand. A more general theory of what DAE systems are consistent can be found in [17] and applied to ODEs in [18].

We use the term **Cauchy convolution** to describe any of the summations from Principles 2.2 through 3.2 that are the most computationally intensive part of every recurrence. So, counting these is one measure of the efficiency of a recurrence algorithm. Some of these can be simplified as shown below, when one array is in a derivative Cauchy product with itself, with examples using these footnote markings in Table 3.1.

$$\begin{aligned} (\dagger) \quad & \frac{1}{k} \sum_{i=1}^{k-1} iW[i] W[k-i] = \frac{1}{2} \sum_{i=1}^{k-1} W[i] W[k-i] \\ (\ddagger) \quad & \frac{1}{k} \sum_{i=1}^k iW[i] W[k-i] = \frac{1}{2} \sum_{i=0}^k W[i] W[k-i] \end{aligned}$$

To derive these, observe that replacing $iW[i] W[k-i]$ with $(k-i)W[i] W[k-i]$ does not change the sum in range 1 to $k-1$ or range 0 to k , and adding both sums gives the simplification. Moreover, the range could then be cut in half by using symmetry, though even and odd cases must be handled differently. One option to make the original more efficient is to store $kW[k]$ beside $W[k]$, sometimes requiring no new computation since $\frac{1}{k}$ is often the last step in computation of $W[k]$. A small disadvantage of $(\ddagger)$ is that it increases the summation range by adding $i=0$, which is significant in the multivariable generalization of these recurrences [20]. (For consistency, every recurrence in Table 3.1 comes from Principles 2.3 or 3.2, but alternative derivations of recurrences with $(\dagger)$ or $(\ddagger)$ follow from directly forming a square without a derivative, as in $w = \sqrt{u}$ is $u = w^2$, and solving for $W[k]$.)

Many standard functions not in Table 3.1 have similar recurrences that require at most two Cauchy convolutions). A particularly worthwhile addition is for $w_1 = \cot(u)$ which is analogous to $\tan$, only inserting a negation. Such minor analogous changes produce recurrences for most other inverse trigonometric and hyperbolic trigonometric

operation	arithmetic DAE	recurrence for $k \geq 1$
$w = \exp(u)$	$w' = w u'$	$W[k] = \frac{1}{k} \sum_{i=1}^k i U[i] W[k-i]$
$w = \ln(u)$	$w' = u'/u$	$W[k] = \left(U[k] - \frac{1}{k} \sum_{i=1}^{k-1} i W[i] U[k-i] \right) / U[0]$
$w = \sqrt{u}$	$w' = (1/2)u'/w$	$W[k] = \left(\frac{1}{2}U[k] - \frac{1}{k} \sum_{i=1}^{k-1} i W[i] W[k-i] \right) / W[0] \quad \dagger$
$w_1 = u^\alpha$	$w'_1 = w_2 u'$	$W_1[k] = \frac{1}{k} \sum_{i=1}^k i U[i] W_2[k-i]$
$w_2 = \alpha u^{\alpha-1}$	$w'_2 = (\alpha-1)w'_1/u$	$W_2[k] = \frac{1}{U[0]} \left((\alpha-1)W_1[k] - \frac{1}{k} \sum_{i=1}^{k-1} i W_2[i] U[k-i] \right)$
$w_1 = \sin(u)$	$w'_1 = w_2 u'$	$W_1[k] = \frac{1}{k} \sum_{i=1}^k i U[i] W_2[k-i]$
$w_2 = \cos(u)$	$w'_2 = -w_1 u'$	$W_2[k] = \frac{-1}{k} \sum_{i=1}^k i U[i] W_1[k-i]$
$w_1 = \tan(u)$	$w'_1 = w_2 u'$	$W_1[k] = \frac{1}{k} \sum_{i=1}^k i U[i] W_2[k-i]$
$w_2 = 1 + w_1^2$	$w'_2 = 2w_1 w'_1$	$W_2[k] = \frac{2}{k} \sum_{i=1}^k i W_1[i] W_1[k-i] \quad \ddagger$
$w_1 = \sin^{-1}(u)$	$w'_1 = u'/w_2$	$W_1[k] = \left(U[k] - \frac{1}{k} \sum_{i=1}^{k-1} i W_1[i] W_2[k-i] \right) / W_2[0]$
$w_2 = \cos(w_1)$	$w'_2 = -w'_1 u$	$W_2[k] = \frac{-1}{k} \sum_{i=1}^k i W_1[i] U[k-i]$
$w_1 = \tan^{-1}(u)$	$w'_1 = u'/w_2$	$W_1[k] = \left(U[k] - \frac{1}{k} \sum_{i=1}^{k-1} i W_1[i] W_2[k-i] \right) / W_2[0]$
$w_2 = 1 + u^2$	$w'_2 = 2u u'$	$W_2[k] = \frac{2}{k} \sum_{i=1}^k i U[i] U[k-i] \quad \ddagger$

TABLE 3.1
Recurrences for nine standard functions

or inverse hyperbolic trigonometric functions. However, any such function referring to sec or csc seems to require three Cauchy convolutions. Hence, it is just as efficient to implement them as in $\sec(u) = 1/\cos(u)$ or $\operatorname{csch}^{-1}(u) = \sinh^{-1}(1/u)$, requiring three Cauchy products including division as one. In fact, Table 3.1 suffices to efficiently write many common functions, such as $\cos^{-1}u = (\pi/2) - \sin^{-1}u$ or $\sinh u = (e^u - e^{-u})/2$, requiring two Cauchy convolutions. Such identities can be called by the software that automates producing recurrences, so that a user may use any of these functions in the original DE function.

Implementing power should use several cases. The Table 3.1 formula for constant exponent requires a nonzero base and uses two Cauchy convolutions. For exponents

2, 3, and 4, repeated multiplication (or squaring) is at least as efficient and allows zero base. Exponent $1/2$ is square root and -1 is a simple division. For variable exponent, using $v^u = \exp(u \ln(v))$ is a good implementation where positive base is needed and the general case requires three Cauchy convolutions. If base v is constant, the scalar multiple results in only two Cauchy convolutions.

4. Generated code for power series solution. The conceptual process of the examples in Section 2 is implemented in PSM4ODES as a MATLAB program `makepsmcode.m` that takes a DE and returns a MATLAB program that generates series coefficients for the series solution. We first look at the use and output of `makepsmcode`, rather than the programming behind the function. This perspective can be useful in a differential equations or numerical analysis course where the focus is on the usefulness of Taylor series solutions. It may be enough to explain that the programming behind the function must interpret the DE file, either by operator overloading (as in explained in Section 5) or by a text file parser, and write the output file of recurrence steps in response.

As is usual for numerical solution, an ODE is written as a system of first-order DEs and given as the vector function f in $y' = f(t, y)$. We call the corresponding MATLAB function a **DE file**, for example `fex1.m` for the function $f(t, y) = \sin(y^2)$ from Example 1. Our program `makepsmcode` takes the DE file and the number of DEs in the system as input and then writes a new file `fex1series.m` with contents below. The call syntax is `makepsmcode(@fex1,1)` where the first input is a function handle. The entire contents of `fex1.m` is echoed in the opening comments of the generated file below (where percentage sign `%` denotes a comment to be ignored in execution). Variable names `t`, `y`, `dydt` in `fex1.m` could be anything but the returned series program will refer to independent `t` and function `y`, and corresponding arrays of coefficients `T` and `Y`. See beginning of [22] for advice and restrictions for forming a compatible DE file.

```
function coefs = fex1series(t0, y0, deg)
%FEX1SERIES finds series coefs for soln of y' = f(t,y) about t0 to deg
% Where f(t,y) was given by:
%   function dydt = fex1(t,y)
%   dydt = sin(y^2);
% Automatically generated by makepsmcode.m, 04-Jun-2024 12:00:40.

numfops = 3; % number of scalar operations used to evaluate f
numys = length(y0);
T = [t0,1,zeros(1,deg-1)];
Y = zeros(numys,deg+1);
U = zeros(numfops,deg+1);
if isa(t0,'sym') || isa(y0,'sym') % allow symbolic, including vpa
    T = sym(T); Y = sym(Y); U = sym(U);
end
Y(:,1) = y0;

% Evaluate f(t0,y0) recording value after each scalar operation
U(1,1) = Y(1,1) * Y(1,1);
U(2,1) = sin(U(1,1));
U(3,1) = cos(U(1,1));
```

```

% Update Y linear coefficient by y' = f(t,y)
Y(1,2) = U(2,1);

% Now recurrence rules for each operation in evaluation of f
for j = 2:deg % j-1 is the power of term at index position j
    U(1,j) = Y(1,j:-1:1) * Y(1,1:j)';
    tempprime = ( U(1,2:j) .* (1:(j-1)) )';
    U(2,j) = (U(3,(j-1):-1:1) * tempprime)/(j-1);
    U(3,j) = -(U(2,(j-1):-1:1) * tempprime)/(j-1);
    % Update Y next coefficient using y' = f(t,y)
    Y(1,j+1) = U(2,j)/j;
end

coefs = Y;
end

```

The algorithm steps in (2.2) are reflected in the "Evaluate" and "recurrence rules" sections in **fex1series**. A number of the implementation choices are specific to MATLAB, where array index origin one is a hassle for translating series index. The arrays $U_i[k]$ and $Y_1[k]$ in (2.2) are implemented in two-dimensional arrays using $U(i, k+1)$ and $Y(1, k+1)$ respectively. Let's explain the more complicated computation of $U(2, j)$, starting with the formula from (2.2) and simply changing the array representation, leaving everything else the same:

$$\begin{aligned}
 U_2[k] &= \frac{1}{k} \sum_{i=1}^k i U_1[i] U_3[k-i] \text{ in index origin zero becomes} \\
 U(2, k+1) &= \frac{1}{k} \sum_{i=1}^k i U(1, i+1) U(3, k-i+1) \text{ in index origin one, or} \\
 U(2, j) &= \frac{1}{(j-1)} \sum_{i=1}^{j-1} i U(1, i+1) U(3, j-i) \text{ using } j = k+1.
 \end{aligned}$$

In any language, the key to efficient series computation is efficient implementation of such Cauchy convolutions. In MATLAB, we compute with vectors of values (instead of a loop on i , or $m = i+1$). We use three vectors for values in the summand, using the MATLAB colon operator: $(1:j-1)$, $U(1,2:j)$, and $U(3,j-1:-1:1)$ where the order is efficiently reversed by $:-1:$. We multiply the components of the first two using $(1:j-1) .* U(1,2:j)$, resulting in coefficients corresponding to u'_1 . We need the dot product of this u'_1 vector with $U(3,j-1:-1:1)$. Timings showed that a dot product (the key to any Cauchy convolution) is efficiently done by matrix multiplication in MATLAB, so one vector times the transpose of the other. We transpose the u'_1 vector into a reusable column vector, leading to the two lines in **fex1series**.

```

tempprime = ( U(1,2:j) .* (1:(j-1)) )';
U(2,j) = (U(3,(j-1):-1:1) * tempprime)/(j-1);

```

Implementing other lines of (2.2) is an exercise in similar techniques. These programming details can be confusing and error-prone by hand. Nevertheless, the automatically generated code can be used and appreciated, without concern for the indexing details.

The output of **makepsmcode**, the function **fex1series** in this example, may be

used directly or called by an ODE solver program. The run `fex1series(0,0.1,25)` returns a row vector of coefficients of the Maclaurin series solution, starting with constant term 0.1 through degree 25 term coefficient $8.6950\text{e-}27$. Each coefficient can be observed to be about one-tenth of the previous term, showing radius of convergence around 10. One may evaluate the degree 25 polynomial using the MATLAB function `polyval` or by programming Horner's rule, as is done in `serieeval` in PSM4ODES. Such evaluations gave the polynomial plots in Figure 2.1. An example value at $t = 8.0$ is 0.4972 returned by `polyval(flip(coefs),8.0)`. One may also compute symbolic series solutions if the MATLAB installation includes the Symbolic Toolbox. Using the `if` statement in the above program, `coefs = fex1series(0,sym('a'),3)` returns `[a, sin(a^2), a*cos(a^2)*sin(a^2), (cos(a^2)*(2*sin(a^2)^2 + 4*a^2*cos(a^2)*sin(a^2)))/6 - (2*a^2*sin(a^2)^3)/3]`.

For a system of multiple DEs, the series program returns a matrix with one row of coefficients for each function in the system solution. For Example 2, consider the second order equation $y'' = \sin(y^2)$ using $y_1 = y$ and $y_2 = y'$ to make the system $y'_1 = y_2$ and $y'_2 = \sin(y_1^2)$. To allow overloading the DE file used by `makepsmcode`, avoid preallocation using `dydt = zeros(2,1)` (see [22]), which may be done as in:

```
function dydt = fex2(t,y)
dydt = [y(2); sin(y(1)^2)];
```

For this second order system, the program `fex2series` written by `makepsmcode` is almost the same as `fex1series` above. The same lines compute `U(2,j)` for $u_2 = \sin(y_1^2)$. Only the "Update Y" lines are changed as in this table.

Update Y for $y' = u_2$	Update Y for $y'' = u_2$
<code>Y(1,2) = U(2,1);</code>	<code>Y(1,2) = Y(2,1);</code>
	<code>Y(2,2) = U(2,1);</code>
<code>Y(1,j+1) = U(2,j)/j;</code>	<code>Y(1,j+1) = Y(2,j)/j;</code>
	<code>Y(2,j+1) = U(2,j)/j;</code>

The output of `fex2series` is a matrix with two rows of series coefficients, for y_1 and y_2 respectively. In this case, the second series is just the derivative of the first, which is how `Y(1,j+1)` is updated. The Horner's rule in `serieeval` can easily evaluate both polynomials by doing computations with columns. Unfortunately, `polyval` does not work on a matrix of coefficients. Of course, this second order differential equation has a significantly different solution than Example 1.

5. Automatic code writing by operator overloading.

5.1. Class properties and variables. We now explain how we use operator overloading to write code for the Taylor series solution of an ODE. We hope some of these concepts and technical details will be interesting, as object-oriented programming or MATLAB software, and helpful for developing such software. The general idea is that evaluation of a DE file for $\mathbf{f}(\mathbf{t},\mathbf{y})$ will do more than just compute numerical value(s), as it would if $\mathbf{t}$ and $\mathbf{y}$ were doubles or $\mathbf{y}$ was an array of doubles. We will define a `trace` class, so that if $\mathbf{t}$ and $\mathbf{y}$ are `trace` objects (or $\mathbf{y}$ is an array of objects), then each operation in evaluating $\mathbf{f}(\mathbf{t},\mathbf{y})$ will call the method for that operation (an overloaded operation) in the class definition file. Each operation does an evaluation of $\mathbf{f}(\mathbf{t},\mathbf{y})$ and, additionally, writes text string(s) for the initialization step(s) and for the recurrence step(s) corresponding to that operation. The strings are then assembled into series program as in the previous section.

The `trace` class traces every numerical step encountered in an evaluation of $\mathbf{f}(\mathbf{t},\mathbf{y})$, keeping track of intermediate values and updating global variables (actually

persistent local variables) needed to generate the corresponding series code. Each **trace** object has three properties at the beginning of the class definition file **trace.m** that begins as follows. In this Section, some code comment lines are stripped out for condensed presentation.

```
classdef trace
    properties
        val % expression value after an operation; or t,y initial
        opcount % index of val in vallist from count_val; or t,y flag
        tlinear % T/F: operations to this point are equivalent to at+b
    end
```

A constructor function will initialize **t** as a trace object with t_0 as **val** and **opcount** 0, flagging the independent variable. Variable **y** is initialized as an array of **trace** objects, where **y(i)** has **val** from the corresponding entry in y_0 vector and **opcount** $-i$, where negative flags the i th component of **y**. In contrast, a positive **opcount** indicates that the object resulted from an operation on **trace** object(s), where the overloaded operation assigned the resulting **val**. This **opcount** comes from updating a persistent local variable **count** (see next paragraph), so that it is the index of the operation auxiliary variable (as in (2.1) for $\sin(y^2)$). Boolean property **tlinear** is not necessary but, if true, allows for writing more efficient code when using the object in a Cauchy convolution, since it has only two nonzero terms in the corresponding series coefficients.

Static class method **count_val** has **persistent** local variables

count: nonnegative integer for the current number of auxiliary operations,
vallist: vector of doubles for consecutive auxiliary operation values,
oplist: array of cells, each is a line of code for auxiliary variable initialization,

reclist: array of cells, each is a line of code for a series recurrence step.

These are initialized to 0, [], {}, {}, respectively, before evaluation of **f(t,y)** by the first call to **count_val** after **clear trace**. An overloaded operation (as discussed in Section 5.2) will send the resulting value and lines of code as arguments to **count_val**, which will increment **count** and augment **vallist**, **oplist**, and **reclist**. We can use the **trace** class directly to show the resulting lists, using steps that mimic the first steps of the driver program **makepsmcode** that is the intended way to use **trace**.

```
>> clear trace
>> t = trace(0.0, 0); % 0 indicates independent variable
>> y = trace(0.9, 1); % 1 indicates dependent function variable
>> h = sin(y^2); % overloaded operations send results to count_val
>> [count, vallist, oplist, reclist] = trace.count_val
count =
    3
vallist =
    0.8100
    0.7243
    0.6895
oplist =
    3x1 cell array
    {'U(1,1) = Y(1,1) * Y(1,1);'}
    {'U(2,1) = sin(U(1,1));' }
    {'U(3,1) = cos(U(1,1));' }
```

```

reclist =
4x1 cell array
{'U(1,j) = Y(1,j:-1:1) * Y(1,1:j).';' }
{'tempprime = ( U(1,2:j) .* (1:(j-1)) ).';' }
{'U(2,j) = (U(3,(j-1):-1:1) * tempprime)/(j-1);' }
{'U(3,j) = -(U(2,(j-1):-1:1) * tempprime)/(j-1);'}

```

These correspond to operations in $\mathbf{f}(\mathbf{t}, \mathbf{y})$ but not necessarily one-to-one. Operation $\mathbf{y}^2$ calls $\mathbf{u}=\mathbf{y}*\mathbf{y}$ (using equivalent parameter names) which produces the first entries on each list. Then, $\mathbf{h}=\sin(\mathbf{u})$ calls $[\mathbf{h}, \mathbf{g}] = \mathbf{sincos}(\mathbf{u})$ which produces two **trace** objects and two calls to **count_val**, updating lists with entries corresponding to *sin* and *cos*. In total, **count** is incremented to 3. Thus, the entries in **vallist** and **oplist** correspond one-to-one with operation auxiliaries that have corresponding arithmetic DAEs and recurrence steps as in Section 2. The example **reclist** shows an additional line to define **tempprime** produced by **sincos**. Other standard functions that require two arithmetic DAEs, such as **tan**, directly produce two **trace** objects and two calls to **count_val** as in Table 3.1 but have only one object output. In contrast, method **sincos** is helpful since both auxiliaries are useful (using same code for calls to **sin** or **cos**). To avoid duplicate code when both outputs are needed, it is possible to use the function **sincos** in the DE file, although built-in MATLAB solvers will not recognize it. One general lesson is that a more efficient DE file will make a more efficient series code, so things like common expressions should be computed and saved to a variable on a separate line. The user's DE file for $\mathbf{f}(\mathbf{t}, \mathbf{y})$ does not have to be just one expression in $\mathbf{t}$ and $\mathbf{y}$ but may use any variable names and any number lines and local variables. Our implementation and generated code, will use $\mathbf{t}$ and write $\mathbf{T}$ for the first argument, $\mathbf{y}$ and $\mathbf{Y}$ for the second argument, and $\mathbf{u}$ and $\mathbf{U}$ for all operation auxiliary variables whether explicitly assigned a name by the user's code or not.

The **opcount** property of each **trace** object plays a critical role in writing the code strings, a task that is accomplished in the overloaded operator methods. In our example, the object $\mathbf{y}$ is passed into the $\mathbf{f}$ function and is used as the actual parameter through operation $\mathbf{y}^2$ to operation $\mathbf{y}*\mathbf{y}$. Object $\mathbf{y}$ carries the -1 **opcount** into this overloaded times operation, so object $\mathbf{y}$ is expressed in strings of the form $\mathbf{Y}(1, \#)$ where $\#$ indicates the appropriate series terms. Based on the current **count** (0 before, 1 after), the output object is given **opcount** 1 and is expressed in strings of the form $\mathbf{U}(1, \#)$, as seen in the first cells of **oplist** and **reclist**. This output object is sent to the **sin** operation and on to **sincos** where it appears as $\mathbf{U}(1, \#)$ in strings. Based on **count** 1 when **sincos** is called, it assigns **opcount** 2 and 3 to the objects corresponding to **sin** and **cos**, respectively, so that these appear in strings as $\mathbf{U}(2, \#)$ and $\mathbf{U}(3, \#)$. Optionally, we use **Static** functions **name** (using **opcount** of the input object), and **nextname** (using **count**), to facilitate writing $\mathbf{U}(\cdot, \cdot)$, $\mathbf{Y}(\cdot, \cdot)$, or $\mathbf{T}(\cdot, \cdot)$, and letting the overloaded code finish with appropriate $\cdot, \#$. Together with all the $\cdot$ marks for **char** strings, these details can be tedious to code or read. One final use of **opcount** enables the update $\mathbf{Y}$ step(s) of the recurrence code. The output of the $\mathbf{f}$ function, with **trace** input, assigns the correct object (including its **opcount**) to each component of the output variable. Our example $\mathbf{h} = \sin(\mathbf{y}^2)$ returned $\mathbf{h}$ with **h.opcount** 2, so the update is $\mathbf{Y}(1, j+1) = \mathbf{U}(2, j)/j$.

The driver program **makepsmcode** initializes $\mathbf{t}$ and $\mathbf{y}$ as **trace** objects and evaluates $\mathbf{f}(\mathbf{t}, \mathbf{y})$. The evaluation calls the overloaded operations in $\mathbf{f}$, and **makepsmcode** calls **count_val** for the number of operation auxiliaries to allocate the $\mathbf{U}$ array, and for

the `oplist` and `reclist` accumulations of code strings. These code strings, together with lines to update `Y`, and other lines that are the same for any `f`, are written into a `fseries.m` file. The program also reads the DE file `f.m` and echoes it into the opening comments.

The input to `makepsmcode` is a function handle `@f` and `numDEs`, which equals the length of the `y` vector, along with optional initial values t_0 and y_0 . If initial values are not specified, the program creates random double values for t_0 and each component of y_0 . This works since evaluations in MATLAB continue operations even with errant values, such as `NaN`, so all the operations in one evaluation `f(t,y)` will be correctly reflected in the series code. Not requiring t_0 and y_0 makes it clear that `makepsmcode` processes the DE and not a specific IVP, assuming there are no branches in the DE file. While tracing a numerical function evaluation is the premise of `makepsmcode`, it turns out that numerical values of input, output, or in `vallist` are really immaterial, and primarily useful for testing the program.

5.2. Class methods that overload operations. Using the `opcount` of `trace` objects as discussed in the previous subsection, we now consider the coding of methods for each of the overloaded operations that may occur in `f`. Specifically, we look at overloading simple negation and overloading `sin` as typical examples. MATLAB Help has a page on "Operator Overloading" that includes a table of operators (using usual symbols) and the corresponding method name that you can overload. Simple negation `-a` has method name `uminus` (for unary minus). Also note that scalar multiplication `a*b` is actually called `mtimes`, since matrix multiplication is the default in MATLAB. Overloading `uminus` is extremely simple but shows the object handling steps that appear in all the overloaded operations.

```
function h = uminus(u)
    %trace/UMINUS overloads negation with a trace object argument
    % h = -u for both operation value and series recurrence code
    newval = -u.val;
    h = trace(newval);
    opstr = [trace.nextname, ',1) = -', trace.name(u.opcount), ',1)'];
    recstr = [trace.nextname, ',j) = -', trace.name(u.opcount), ',j)'];
    h.opcount = trace.count_val(newval, {opstr}, {recstr});
    h.tlinear = u.tlinear;
end
```

An example call to `uminus` might be in the expression `-sin(y^2)`, adding just one negation to our Example 1. The call to `sin` will return a `trace` object, with `opcount` 2, which becomes the actual parameter input for formal parameter `u` of `uminus`. The persistent class variable `count` will be 3 when `uminus` called (since a `cos` object is also created). The `uminus` call creates a new `trace` object, locally called `h`, which will be returned as the output of `uminus`. The class constructor `trace` sets the `val` property of `h`, but other properties need to be updated. In our example call, `opstr` would become the text string `'U(4,1) = -U(2,1);'` using `trace.name(u.opcount)` which returns string `'U(2'` and `trace.nextname` which returns `'U(4'`. Method `nextname` calls `trace.count_val` without any argument to get `count`. However, `count` is not incremented until `trace.count_val` is given the next items to append to the persistent lists, as on the second to last line of `uminus`. Notice that `opstr` and `recstr` are put into a cell by using braces, allowing different length strings to

be appended to the vertical array of cells `oplist`. In different example calls `-t` and `-y(2)`, `trace.name(u.opcount)` would produce `'T(1'` and `'Y(2'`, respectively for `u.opcount` values 0 and `-2`.

With `uminus` providing a template for handling the objects, strings, and lists, the overloading of other functions primarily involves creating strings for the recurrence relations in Table 3.1. When a pair of recurrences are required (needing w_1 and w_2 in the table), the code will define two objects and append both associated persistent list items, but usually only the first (requested) object is returned. The `sincos` method returns both objects.

```
function [s, c] = sincos(u)
%trace/SINCOS returns trace objects for sin(u) and cos(u)
% s = sin(u) and c = cos(u)
% s' = c * u' and c' = -s * u' Cauchy products series recurrence
newval = sin(u.val);
s = trace(newval);
sname = trace.nextname;    % 'U(i' where i is trace.count_val + 1
cname = trace.nextname(2); % 'U(i' where i is trace.count_val + 2
uname = trace.name(u.opcount);
opstr = [sname, ',1) = sin(', uname, ',1));'];
if u.tlinear
    reccell = {[sname, ',j) = ', cname, ',j-1)*', uname, ',2)/(j-1);'];
else
    % tempprime pulls coefs for u' out of array coefs for u
    recstr1 = ['tempprime = ( ', uname, ',2:j) .* (1:(j-1)) ).'';'];
    recstr2=[sname, ',j) = (', cname, ',(j-1):-1:1) * tempprime)/(j-1);'];
    reccell = {recstr1;recstr2};
end
s.opcount = trace.count_val(newval, {opstr}, reccell);
s.tlinear = false;

newval = cos(u.val);
c = trace(newval);
opstr = [cname, ',1) = cos(', uname, ',1));'];
if u.tlinear
    recstr = [cname, ',j) = -', sname, ',j-1)*', uname, ',2)/(j-1);'];
else
    recstr=[cname, ',j) = -(', sname, ',(j-1):-1:1) * tempprime)/(j-1);'];
end
c.opcount = trace.count_val(newval, {opstr}, {recstr});
c.tlinear = false;
end
```

This code constructs trace object `s` and, after the string writing steps, updates `s.opcount` by a call to `trace.count_val(newval, {opstr}, reccell)` which appends to globals `vallist`, `oplist`, and `reclist`. For the example that creates the lists of Subsection 5, the above code first assigns `opstr` to be the string `'U(2,1) = sin(U(1,1))'` and then assigns `reccell` to be a cell array of two strings for the lines defining `tempprime` and `U(2,j)`. Similarly, the trace object `c` is constructed and

creates lines for $U(3,1)$ and $U(3,j)$, reusing `tempprime`. Notice how `u.tlinear` will simplify the code if the argument `u` is linear in t , as in example `sin(3*t+1)`, where `tempprime` is unnecessary since the Cauchy convolution of Principle 2.3 would have only one nonzero term. For other operations, such as multiplication, `u.tlinear` could differently handle the case of a Cauchy convolution with only two nonzero terms (as in the preliminary example of Section 2), although we do not implement it for simplicity and since little time improvement was noticed in MATLAB.

The overloaded function `sin` simply calls `sincos` and returns only the first object.

```
function h = sin(u)
    %trace/SIN overloads sine with a trace object argument
    [h, g] = sincos(u);
end
```

Similarly, `cos` returns `g`.

We can overload other functions that are defined in terms of just the few given in Table 3.1, as in the following.

```
function h = csc(u)
    %trace/CSC overloads csc of a trace object argument
    h = 1/sin(u);
end
```

Using `csc` will result in code for three Cauchy convolutions by calling `sincos` indirectly and `mrdivide` which overloads the division with one Cauchy convolution defined by Principle 3.1. This brings up another issue in overloading many functions: there are simpler special cases.

When overloading a function with two arguments, including `plus`, `minus`, `mtimes`, `mrdivide`, and `mpower`, we separately program the case when one of the arguments is not a trace object but is numerical (as in `1/sin(u)` above). In defining trace method `mtimes(u,v)`, we have three cases: `u` is not a `trace`, `v` is not a `trace`, and else general case which is the only one that requires a Cauchy product. Note that if both arguments are numerical, the overloaded trace function will not be called but the numerical value is computed as usual. For example `p=3; dydt = 2*p*y;`, will show up in series code same as `6*y` would. We use `num2str(u,18)` to write the double value with 18 digits, to ensure recovery of the corresponding machine number (even though this may look like it introduces roundoff error in constants). Method `mpower(u,r)` uses several special cases as discussed at the end of Section 3. We've frequently used `u^2`, where `mpower` simply calls `u*u`, and thus `mtimes`, in this case. If `r` is variable, determined by `isa(r,'trace')`, then `mpower` calls `exp(r*log(u))`, calling three overloaded functions that result in three Cauchy convolutions, unless `u` is numerical and then only two overloaded functions are called. The only Cauchy convolution directly programmed in `mpower` is when `r` is a non-special numerical constant that uses the u^a pair of recurrences in Table 3.1.

As alternatives to the implementation of this paper, we have programmed two different approaches to using object-oriented computing for an algorithmic Taylor series solution of ODEs. Both approaches preceded and informed this work but are not used in PSM4ODES. The simplest approach is found in [19] where any function u is an instance of a `series` class that holds the vector of series coefficients to some degree d . Operations on series objects will perform the associated recurrence in Section 3. For a function of $f(t)$, the series object for t is known to degree d , so one overloaded evaluation of f will return the series coefficients for $f(t)$. This implementation is recommended for finding Taylor series for function expressions.

However, it is inefficient in solving $y' = f(t, y)$. If coefficients for y are known to only degree k (initially $k = 0$), then overloaded evaluation of $f(t, y)$ will return the coefficients for y' which gives the next coefficient of degree $k + 1$. The overloaded evaluation must happen inside a loop on degree k , which does wasteful recalculation of all the lower order series coefficients for y and all the intermediate operations that are not saved anywhere. The second alternative approach saves these intermediate series values for a class that performs the overloaded operations instead of writing them. This implementation used two classes for overloading, **valtrace** and **sertrace**. The first evaluation of $f(t, y)$ uses valtrace objects much as in trace, but counting and storing (in vallist) only the numerical value at each step (nothing about higher series terms). The sertrace class can be used in the looping just described for the first alternative but without the wasted recalculation. A persistent handle (or pointer) to one array **uall** of sertrace objects, one for each of the steps (enumerated by valtrace), plays the role of the array **U** in code written by **makepsmcode**. Evaluation of $f(t, y)$ is done inside a loop on degree k , where each overloaded operation computes the next coefficient using values in **uall**. This performs exactly the same floating point computations as would code generated by **makepsmcode**. However, it is slower due to the overhead of all the calls and references. The generated recurrence code would be even more efficient in a compiled language.

6. DE solvers using high-order Taylor series. Generated code for series recurrence can easily be used for numerically solving the associated differential equation using a high-order Taylor series method. The step size through time can be fixed or variable as in most numerical solvers. The order of the solution method (the degree to which the series is computed) can also be chosen by the user, chosen by the program, or variable along with step size. We begin with a simple fixed step size **h** example.

6.1. Fixed step size solver odepsmh. The program **odepsmh** is an explicit numerical solver that repeatedly uses (t_k, y_k) to compute (t_{k+1}, y_{k+1}) (as in all one-step numerical ODE solvers). Here, the next value y_{k+1} is computed from the Taylor series about t_k evaluated at t_{k+1} . For a DE file, **makepsmcode** need be run only once. This solver will expect or create series recurrence code, say **fex1series.m**, to match the DE file, respectively **fex1.m**, given by input function handle. Note: if you change a DE file contents, you must delete the corresponding series file, so the correct one will be created. The series recurrence code is given the starting value of (t_k, y_k) and degree **deg**, and returns series coefficients Y from 0 to **deg**. The polynomial evaluation $y_{k+1} = \sum_{j=0}^{\text{deg}} Y[j](t_{k+1} - t_k)^j$ is most efficiently done by Horner's rule, done by calling **serieseval** in **odepsmh** code below. Each y_k can be a column vector, so we use array **y** where each column **y(:,k)** are values of all components of y at one time. The series coefficients are held in matrix **coefs** which is the Y matrix computed by series recurrence code, so that **coefs(:,m)** holds coefficients for all components of degree $m - 1$. Index origin one requires the offset index from 1 to **deg + 1**. We evaluate all components of y using column arithmetic.

```
function [t,y] = odepsmh(fhandle,tspan,h,y0,deg)
t0 = tspan(1);
tend = tspan(2);
if h*(tend - t0) < 0; h = -h; end % match sign of h to direction
t = (t0:h:tend).'; % t is a column vector of times t(k), with t(1)=t0
if t(end) ~= tend
```

```

    t = [t;tend];
end
numpts = length(t);

% series recurrence code is generated if it doesn't already exist
seriesfuncname = [func2str(fhandle),'series'];
if isfile([seriesfuncname,'.m'])
    fseries = str2func(seriesfuncname); % return function handle
else
    fseries = makepsmcode(fhandle,length(y0));
end

y = zeros(length(y0),numpts); % index by y(component, time)
y(:,1) = y0;
for k = 1:(numpts-1)
    coefs = fseries(t(k),y(:,k),deg);
    y(:,k+1) = serieeval(coefs,t(k),t(k+1));
end
y = y.'; % transpose into output form

```

We test `odepsmh` on another example of a forced damped pendulum model that can be chaotic. The equation

$$y'' = -\sin(y) - 0.1y' + \cos(t)$$

is given in the DE file:

```

function dydt = fdpendulum(t,y)
    dydt = [ y(2); -sin(y(1)) - 0.1*y(2) + cos(t)];

```

We start the pendulum at the bottom and give it a strong kick using $y(0) = 0, y'(0) = 2$. The call `[tpsm,ypsm] = odepsmh(@fdpendulum,[0,200],0.6,[0;2],20)` uses large step size $h = 0.6$ and degree 20 series to compute an accurate solution through time 200. A phase plane plot of the solution, labeled `odepsm h = 0.6`, is shown in Figure 6.1. We compare three other numerical solutions that show the sensitiv-

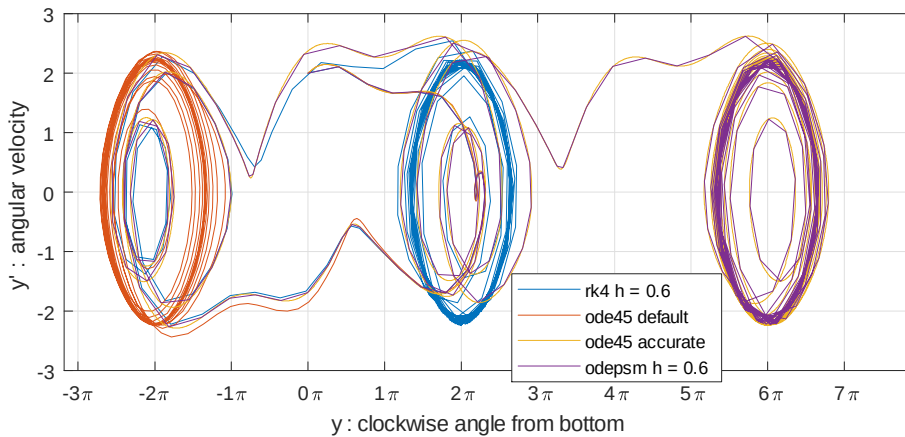


FIG. 6.1. *Different numerical solutions of a forced damped pendulum*

ity. The phase plane shows different "wells" around even multiples of π where the pendulum can oscillate until it goes over the top (with positive or negative velocity) into a different well. The `ode45` default curve shows that a call to MATLAB's `ode45(@fdpendulum,[0,200],[0;2])` with default options does not find the correct well. Neither does the rk4 curve, the classic Runge-Kutta order 4 with fixed step size of $h = 0.6$. The `ode45` accurate curve was generated by `ode45` using the smallest allowed relative tolerance of **2.22045e-14**, which is also used as the absolute tolerance. The `odepsm` curve appears as chords that cut across the `ode45` accurate curve, not because of inaccuracy but because of the large step size. The `odepsmh` code plots only endpoints of each 0.6 time span, each computed using degree a degree 20 Taylor polynomial.

The following Table 6.1 shows efficiency and accuracy of the comparative numerical solution runs, with the first four rows being the runs reflected in Figure 6.1, and the last two using the adaptive step size program `odepsmJZ` described in Section 6.2 below. Reported errors are the absolute error in the ending value of this sensitive system, measured against a hard-coded ending value with about 20 significant digits found using variable precision arithmetic (`vpa`) in `odepsmh`.

	num steps	mean step	y(end) value	abs error	relative time
Runge-Kutta 4	334	0.6000	4.85351621078251	1.3e+01	1.5%
ode45 default	256	0.7813	-7.71282407459629	2.5e+01	4.3%
ode45 accurate	28778	0.0069	17.41704528241961	3.1e-12	100.0%
odepsmh deg 20	334	0.6000	17.41704249607110	2.8e-06	36.0%
odepsmh deg 20	800	0.2500	17.41704528241632	1.5e-13	74.5%
odepsmJZ deg 17	798	0.2506	17.41704528241637	9.9e-14	72.1%
odepsmJZ deg 5	690	0.2899	17.42184618980130	4.8e-03	24.9%

TABLE 6.1
Performance of solvers on a forced damped pendulum

With only 334 steps, `odepsmh` has absolute error on the order of 10^{-6} . The first two rows show the unsuccessful results despite a similar number of steps. The `ode45` accurate row (requesting tolerances of **2.22045e-14**) had error of order 10^{-12} found using 28,778 steps, and was used as the standard for relative time. (The step counts adjust for the default refinement factor of 4, so that the `ode45` default curve plotted 1024 points for 256 steps of the solver. The `ode45` accurate curve used option `Refine=1` to only return solver steps.) Rows 5 and 6 show `odepsm` computations that achieve better than the "accurate" value in a little less time than `ode45`, using $h = 0.25$ fixed step size in `odepsmh` and requesting tolerance of **2.22045e-14** in `odepsmJZ`. The last row succeeds in finding enough accuracy for a reliable plot (where `ode45` default fails), by calling `odepsmJZ` with tolerance **1e-3** which used degree 5 Taylor polynomials and 690 steps. The `timeit` function in MATLAB is used to average timings over multiple calls (on an old Windows 11 laptop running MATLAB 2025b). Since timings can vary by machine and status, the timings are reported as a percentage of the standard. Only the relative magnitude of these timings seem dependable and timing results in Section 7 are less flattering. Timings for the `odepsm` programs do not include the time to generate `fdpendulumseries`, since this is needed only once for a DE file. The one run of `makepsmcode` to generate the series code took around 75% relative time, using the same standard. Clearly, `odepsm` methods can achieve accuracy in this

example using vastly fewer steps than `ode45`, in better, or at least comparable, time.

6.2. Adaptive step size solver `odepsmJZ`. When using series recurrence code in an adaptive solver, there are more ways to adjust based on error tolerance than in standard adaptive programs. In addition to adjusting step size, we can adjust the order of series terms to be generated, either once using tolerance or dynamically based on error estimates. Also, series coefficients can be used to estimate error and/or estimate radius of convergence that informs step size choice. In `odepsmJZ`, we implemented a version of the adaptive algorithms used by Jorba and Zou in [13].

First, we sketch the principles used from [28] that try to optimize efficiency within error tolerance ε , thinking asymptotically and ignoring constants. On each step, compute series terms to degree N , estimate the radius of convergence ρ , and choose step size h as a fraction (less than half) of ρ . Assuming the series is asymptotically geometric, next value $y_{k+1} = \sum_{j=0}^N Y[j](t_{k+1} - t_k)^j$ has error that is bounded by the last term that is roughly equivalent to $\rho^{-N} h^N$. Set this equal to ε and solve for $N = \ln(\varepsilon)/\ln(h/\rho)$. The cost of computing the series coefficients on each step is assumed to be proportional to the cost N^2 of one Cauchy product. The number of steps is proportional to $1/h$, so we wish to minimize N^2/h , or maximize h/N^2 . Substituting for N in terms of h , it suffices to maximize $h(\ln(h/\rho))^2$. Calculus finds the maximum at $h = \rho/e^2$, fixing the fraction of ρ . Using this h , gives $N = -\ln(\varepsilon)/2$ that depends only on the tolerance.

Our adaptive code follows [13] with practical adjustments to this theoretical N and h . Assume, for simplicity, that one parameter `tolerance` is used to request the same absolute and relative errors. The implemented N is one more than theory and uses the ceiling function:

```
deg = ceil(1-log(tolerance)/2).
```

So, requested tolerances `1e-13` and `1e-3` use degrees 16 and 5, respectively.

Our program starts with the header

```
function [t,y,deg] = odepsmJZ(fhandle,tspan,y0,tolerance)
```

and has much the same structure as `odepsmh`. The for loop becomes a while loop with the choice of step size `h` inserted after computing

```
coefs = fseries(t(k),y(:,k),deg)
```

and before updating `y(:,k+1)` using Horner's rule. For each component `i` of the solution, the value at `t(k)` is `y(i,k)` and the Taylor polynomial solution $\sum_{j=0}^{\infty} a_j(t - t_k)^j$ has coefficients $a_j = \text{coefs}(i,j+1)$. We follow [13], which uses an adaptation of the root test for series, which gives radius of convergence $\rho = \lim_{j \rightarrow \infty} (1/a_j)^{1/j}$. First, maximize across components of `y`, to make a scaling factor s , and across the coefficients of each component, to one make representative a_j for all `i`. Then, compute one $\rho_j = (s/a_j)^{1/j}$ for each j from 1 to `deg` (in our vector `roottests`).

```
scale = max(1,norm(y(:,k),inf));
maxcoefs = max(abs(coefs(:,2:end)),[],1);
roottests = (scale./maxcoefs).^(1./(1:deg));
hm = min(roottests([deg-1,deg]))/exp(2) * exp(-0.7/(deg-1));
h = min(hm, min(roottests));
```

In place of the theoretical limit ρ in $h = \rho/e^2$, `hm` uses the minimum of the last two ρ_j terms (and factor $1/e^2 \approx 0.135$). The factor $\exp(-0.7/(deg-1))$ is a further step reduction for safety, a factor that ranges from 0.7 to 1 for `deg` > 2. Using the last two terms protects against a series where every other a_j could be very small (or zero), leading to an unreasonably large step. In the case that `hm` is still large compared to

earlier ρ_j terms (e.g. a nearly quadratic solution), the final safety is to take at least the minimum of all computed ρ_j terms.

This is just one example of an adaptive scheme that uses generated series code. Table 6.1 shows good success in matching the requested tolerances (relative and absolute) of $2.22045\text{e-}14$ and $1\text{e-}3$.

7. Performance of odepsmJZ on test set problems. We use work-precision diagrams to compare the odepsmJZ solver to several solvers provided by MATLAB, on the forced damped pendulum of the previous section, and then on a couple of problems from the *Test set for IVP problems* in [15] (developed in [14]). The test set problems have published reference solutions at the end of an integration interval, enabling error vector computations. Precision is reported as significant correct digits (scd) computed by $-\log_{10}(\|\text{relative error}\|_\infty)$. Each run of a solver for a specified tolerance will result in one point in the work-precision diagram, plotting the resulting scd on horizontal axis and CPU execution time on the vertical. Execution time is computed by the `timeit` function which averages over multiple calls, although execution times can still vary over different plots. The execution time for odepsmJZ does not include the time to generate the series code once for the DE; this time is shown as a horizontal line labeled "psm coding." Each solver produces a curve of points, one for each requested tolerance, equal relative and absolute, of $2.3 \cdot 10^{-m}$ for $m = 3, 4, \dots, 14$ (so last exceeds minimum error of $2.22045\text{e-}14$ allowed by MATLAB's solvers), which gives scd goals of about $m - .36$ for each tolerance. Numerical point labels are the order used by odepsmJZ depending on tolerance.

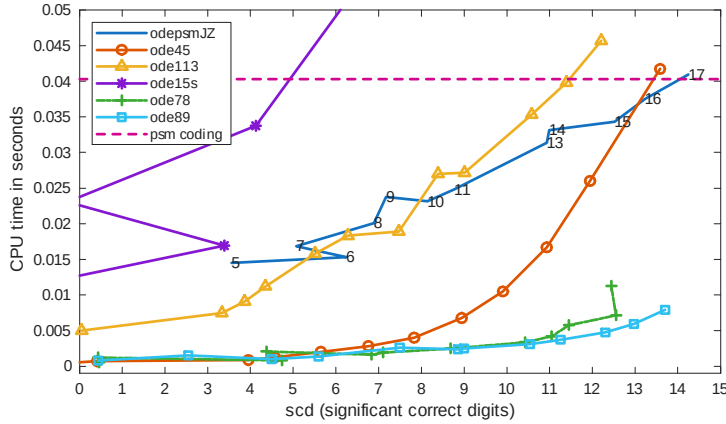


FIG. 7.1. Work-precision diagram for the forced damped pendulum problem. (Numerical point labels are the order used by odepsmJZ.)

Figure 7.1 is a work-precision diagram for the forced damped pendulum. While this pendulum problem is sensitive to small changes in initial (or computed) values, it is not a stiff equation, so `ode15s` is a wrong choice but is only included for consistency of examples. For efficiency, test examples of this section show that `odepsmJZ` is usually slower than the MATLAB solvers, although it is competitive in this example compared to `ode113` and, at the highest precision, `ode45`. Considering accuracy, a point on the `odepsmJZ` curve is usually to the right of the corresponding point on each of the other curves, indicating more precision. For this pendulum problem, `odepsmJZ` is the only solver that meets or exceeds the scd goal corresponding to every tolerance

setting, although it comes close at the point labeled with order 11 with 8.64 scd goal. (These observations are clearer in a graphic similar to Figure 7.6 in [22].) We see that `ode89` is also very accurate, as well as being fast.

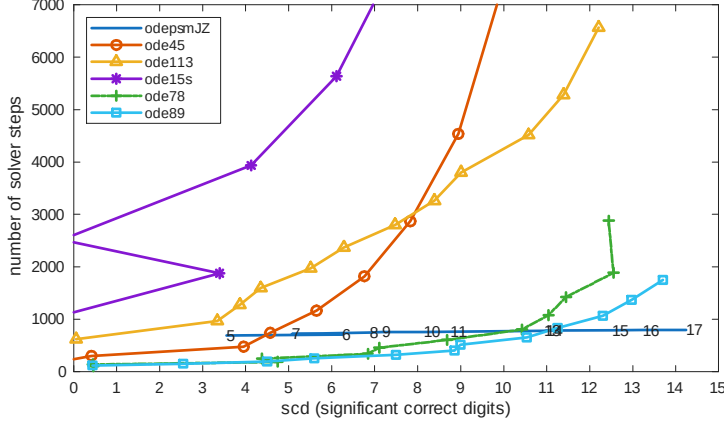


FIG. 7.2. Number of solver steps for forced damped pendulum at different tolerances.

In Figure 7.2 we plot number of solver steps versus scd, to call attention a distinctive feature of `odepsmJZ`. Unlike other methods that achieve higher scd by increasing the number of steps, `odepsmJZ` keeps the number of steps relatively flat (not perfectly), by instead increasing the order n of the Taylor polynomial used. A spline of n th-order Taylor polynomials is computed for this number of steps. This may have implications for the representation of a high-precision solution of the ODE. Runge-Kutta methods use DE evaluations to approximate error(s) of fixed order(s), as in orders 8 and 9 for `ode89`, albeit with efficient computation and interpolation methods. While our MATLAB implementation of this high-order Taylor method does not save time over these refined solvers, other implementations (e.g. [1] [3] [17] [13]) have been very successful. Our executable series recurrence of the ODE may clarify this distinctive approach.

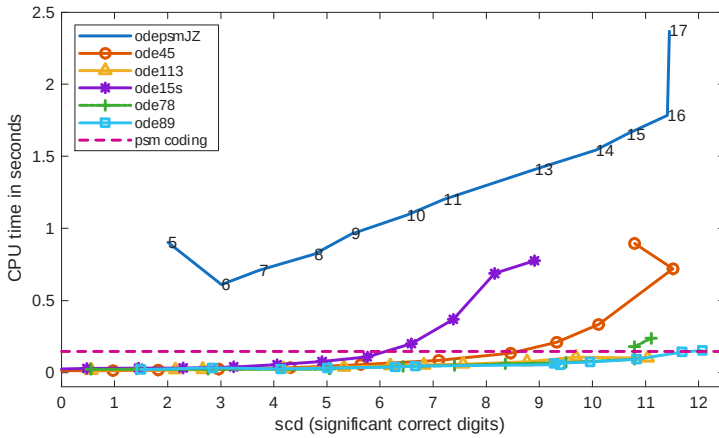


FIG. 7.3. Work-precision diagram for the Pleiades celestial mechanics problem.

Figure 7.3 is a work-precision diagram for the Pleiades celestial mechanics prob-

lem, the only nonstiff equation in the test set. Again, `odepsmJZ` is slower, but competes with `ode89` for most precise, both being about one scd short of the goals set by tolerances through all but the last tolerance, where `ode89` improves and `odepsmJZ` does not. The plot of the number of steps (in [22]) looks very similar to Figure 7.2.

To generate the series code using our software, the ODE system for the Pleiades problem must be coded with scalar operations. The problem models 7 stars with (x_i, y_i, x'_i, y'_i) position and velocity, for a 28 dimensional system. For each star i , the acceleration equation must sum over the effect (12 operations with $^{3/2}$ being the only beyond basic arithmetic) from every other star j , a natural nested loop. Each of the solvers uses this same simple scalar code with nested loop as the DE file. For each of the $7 \times 6 \times 12 = 504$ operations in the system, our software will write one line in the series code to initialize a U variable and write at least one other line for its recurrence step. The result is a series code with 1279 lines, including $7 \times 6 \times 6 = 252$ Cauchy convolutions. This series code runs quickly but demonstrates how the simple concept of a recurrence for each operation is tied to scalar operations. A search of MATLAB Help Center for "Solve Pleiades" returns a page with a `pleiades` function that uses outer products to compute all combinations i and j of a vector to compute matrices of results instead of using a nested loop, a nice use of MATLAB's strengths. Our trace class could overload such operations by running a nested loop and generating the same lines of series recurrence code, but this seems disingenuous. A true reflection of the MATLAB structure in the series code is beyond the scope of this paper. Writing and processing scalar DE files emphasizes our approach to generating series code and is applicable in all computing languages.

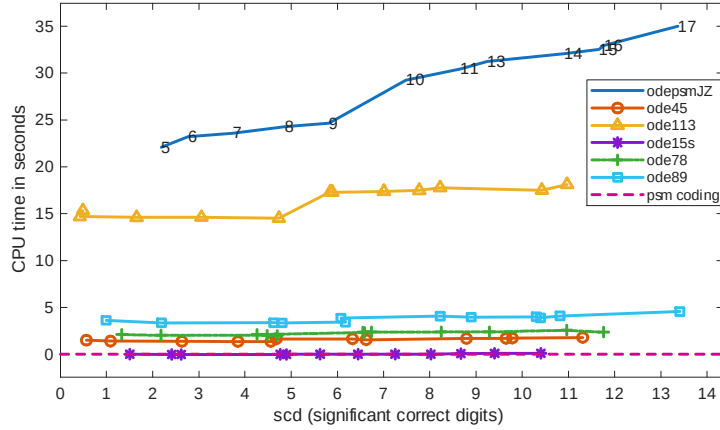


FIG. 7.4. Work-precision diagram for the van der Pol oscillator problem.

Figure 7.4 is a work-precision diagram for the van der Pol oscillator problem, a stiff DE for the parameter $\mu = 1000$. Again, `odepsmJZ` is slower but usually most precise. The stiff solver `ode15s` is by far the fastest, but is usually 2 to 3 digits short of the scd goal. All of the MATLAB solvers have surprisingly flat execution times across tolerances. This corresponds to using a fairly flat number of steps in the millions (computing to time 2000) as shown in Figure 7.5. There we see that `odepsmJZ` again results in a different strategy, this time by reducing the numbers steps as the order increases. To better visualize the precision performance in Figure 7.6, each scd goal (given by tolerances) is the horizontal coordinate, and the number of significant digits by which the solver exceeds (or comes short of) that scd goal

is on the vertical. While `odepsmJZ` is the most precise for the higher `scd`'s, both `ode89` and `odepsmJZ` come very near to the highest goal (13.64). Over all solvers and tolerances, only `odepsmJZ` meets an `scd` goal which only occurs using orders 11, 14 and 15 respectively.

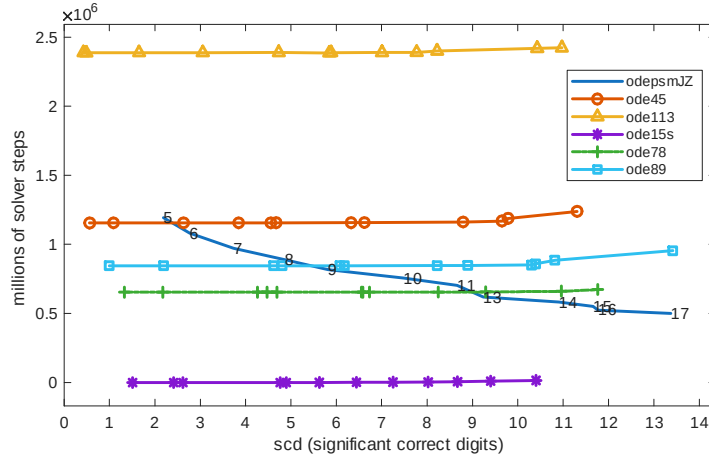


FIG. 7.5. Millions of solver steps for the van der Pol oscillator problem.

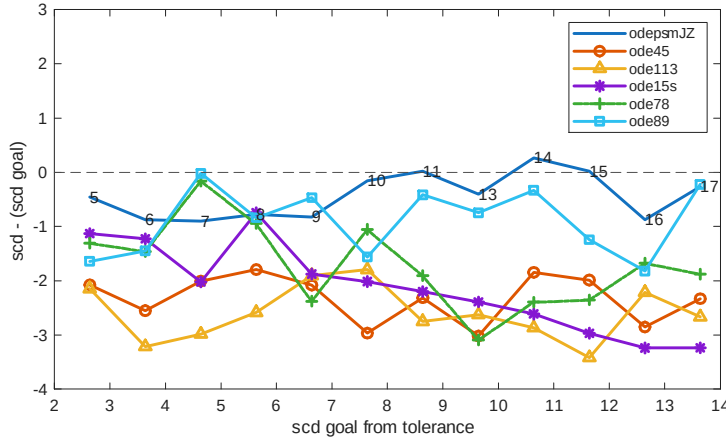


FIG. 7.6. Number of significant correct digits above (or below) the `scd` goal given by tolerance for the van der Pol oscillator.

8. Conclusion. Anyone studying or applying differential equations should be aware that a series recurrence relation algorithm is available for the solution of almost any ODE initial value problem. Finding the recurrence is methodical and a computer program can produce executable code for the recurrence around a variable initial point to variable order. In a differential equations class, students can see that the idea of a power series solution is not restricted to simple textbook examples of linear equations. Using the recurrence code, the radius of convergence can be estimated from the series coefficients and visualized by plotting different order Taylor polynomial solutions. When considering numerical methods, students or applied mathematicians can see

the practicality of the Taylor series method to any desired higher order. Tests show that high-order methods can be competitive with other methods and MATLAB's ode45. The high-order methods are particularly useful in applications that require high accuracy [2].

The accompanying PSM4ODES software (also available via [21]) carries an open source license and may be used, studied, and modified as is helpful, with educational uses especially encouraged. A parallel exposition and development in Python, Julia, or C++ could reach other audiences. Other developers can develop (and have developed) professional tools that fine tune efficiency based on language and computing environment. There is potential for further research into adaptive methods that can dynamically adjust the order used in addition to the step-size. One idea is using the *a priori* error bound; we've used a version of the trace class that also returns constants in the final bound in [30]. There seems to be boundless potential for the algorithmic Taylor method in solving ODEs.

REFERENCES

- [1] A. Abad, R. Barrio, F. Blesa, M. Rodriguez, *Algorithm 924: TIDES, a Taylor Series Integrator for Differential EquationS*, ACM Trans. Math. Softw. 39 (2012), pp. 5:1-28, doi.org/10.1145/2382585.2382590.
- [2] D.H. Bailey, R. Barrio, J.M. Borwein, *High-precision computation: Mathematical physics and dynamics*, Applied Mathematics and Computation, 218 (2012), pp. 10106-10121, doi.org/10.1016/j.amc.2012.03.087.
- [3] F. Biscani, D. Izzo, *Revisiting high-order Taylor methods for astrodynamics and celestial mechanics*, Monthly Notices of the Royal Astronomical Society, 504 (2021), pp. 2614–2628, doi.org/10.1093/mnras/stab1032.
- [4] A. Borri, F. Carravetta, and P. Palumbo, *Quadratized Taylor series methods for ODE numerical integration*, Appl. Math. Comput., 458 (2023), p. 128-237, doi.org/10.1016/j.amc.2023.128237.
- [5] W. E. Boyce and R. C. DiPrima, *Elementary Differential Equations and Boundary Value Problems*, 10th ed., Wiley, Hoboken, NJ, 2012.
- [6] R. L. Burden and J. D. Faires, *Numerical Analysis*, 8th ed., Thomson Brooks/Cole, Belmont, CA, 2005.
- [7] Y. F. Chang and G. Corliss, *ATOMFT: Solving ODEs and DAEs Using Taylor Series*, Computers & Mathematics with Applications, 28 (1994), pp. 209-233.
- [8] G. Corliss and Y. F. Chang, *Solving Ordinary Differential Equations Using Taylor Series*, ACM Trans. Math. Softw., 8 (1982), pp. 114–144, doi: 10.1145/355993.355995.
- [9] S. A. Forth, *An Efficient Overloaded Implementation of Forward Mode Automatic Differentiation in MATLAB*, ACM Transactions on Mathematical Software, 32 (2006), pp. 195–222, doi.org/10.1145/1141885.1141888.
- [10] A. Gibbons, *A Program for the Automatic Integration of Differential Equations using the Method of Taylor Series*, Computer J., 3 (1960), pp. 108-111, doi: 10.1093/comjnl/3.2.108.
- [11] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed., SIAM, Philadelphia, PA, 2008.
- [12] J. Guenther and M. Wolf, sponsor P. Warne, *An Adaptive, Highly Accurate and Efficient, Parker-Sochacki Algorithm for Numerical Solutions to Initial Value Ordinary Differential Equation Systems*, SIAM SUIRO, 12 (2019), doi: 10.1137/19S019115.
- [13] A. Jorba and M. Zou, *A Software Package for the Numerical Integration of ODEs by Means of High-Order Taylor Methods*, Experimental Math., 14 (2005), pp. 99-117, doi: 10.1080/10586458.2005.10128904.
- [14] W.M. Lioen and J.J.B. de Swart, *Test set for initial value problem solvers*, (1998), Modelling, Analysis and Simulation [MAS], CWI, <https://ir.cwi.nl/pub/4619>.
- [15] F. Mazzia and F. Iavernaro, *Test set for initial value problem solvers*, Release 2.4 February 2008, Department of Mathematics, University of Bari, Italy, <https://archimede.uniba.it/~testset/report/testset.pdf>.
- [16] R.E. Moore, *Methods and Applications of Interval Analysis*, SIAM, 1979, Section 3.4.
- [17] N.S. Nedialkov and J.D. Pryce, *Solving Differential-Algebraic Equations by Taylor Series (I): Computing Taylor Coefficients*, BIT Numerical Mathematics 45 (2005), pp. 561-591, doi:

- 10.1007/s10543-005-0019-y.
- [18] N.S. Nedialkov and J.D. Pryce, *Sub-ODEs Simplify Taylor Series Algorithms for Ordinary Differential Equations*, preprint to appear in SIAM J. on Scientific Computing, arXiv:2503.21078.
 - [19] R.D. Neidinger, *Introduction to automatic differentiation and MATLAB object-oriented programming*, SIAM Rev. 52 (2010), pp. 545–563.
 - [20] R.D. Neidinger, *Efficient recurrence relations for univariate and multivariate Taylor series coefficients*, AIMS Conference Publications, 2013, 2013(special): 587–596. doi: 10.3934/proc.2013.2013.587.
 - [21] R.D. Neidinger, *PSM4ODES*, <https://github.com/rineidinger/psm4odes>.
 - [22] R.D. Neidinger, *User Manual for PSM4ODES*, part of the PSM4ODES software package.
 - [23] G.E. Parker & J.S. Sochacki, *Implementing the Picard Iteration*, Neural, Parallel and Scientific Computation 4 (1996), 97–112.
 - [24] M.A. Patterson, M. Weinstein, A.V. Rao, *An efficient overloaded method for computing derivatives of mathematical functions in MATLAB*, ACM Transactions on Mathematical Software, 39 (2013), pp. 1–36, doi.org/10.1145/2450153.2450155
 - [25] J.D. Pryce, *A simple structural analysis method for DAEs*, BIT Numerical Mathematics 41 (2001), pp. 364–394.
 - [26] L.B. Rall, *Automatic Differentiation: Techniques and Applications*, Lecture Notes in Computer Science 120, Springer-Verlag, Berlin, 1981.
 - [27] C. Rackauckas and Q. Nie, *Differential Equations.jl – a performant and feature-rich ecosystem for solving differential equations in Julia*, J. of Open Research Software 5 (2017), OrdinaryDiffEqTaylorSeries package <https://docs.sciml.ai/DiffEqDocs/stable/api/ordinarydiffEq/explicit/TaylorSeries/>.
 - [28] C. Simó, *Global Dynamics and Fast Indicators*, in Global Analysis of Dynamical Systems, H.W. Broer, B. Krauskopf, and G. Vegter, eds., IOP Pub., Bristol, 2001, pp. 373–389.
 - [29] J. Sochacki, A. Tongen, *Applying Power Series to Differential Equations*, Springer, Cham, Switzerland, 2022.
 - [30] R.J. Thelwell, P.G. Warne, D.A. Warne, *Cauchy-Kowalevski and polynomial ordinary differential equations*, Electron. J. Diff. Equ. (2012), pp. 11:1–8.
 - [31] J. Waldvogel, *Der Tayloralgorithmus*, J. Applied Math. and Physics (ZAMP) 35 (1984), 780–789.
 - [32] P.G. Warne, D.A. Polignone Warne, J.S. Sochacki, G.E. Parker, and D.C. Carothers, *Explicit A-Priori error bounds and Adaptive error control for approximation of nonlinear initial value differential systems*, Computers & Math, with Appl., 52 (2006), pp. 1695–1710, doi: 10.1016/j.camwa.2005.12.004.